Model Architecture Summary

Kolkata House Price Predictor

Extra Trees Regression Pipeline · Source: cleaning_data.ipynb · Dataset: House_Price.csv

1. Dataset

House_Price.csv is loaded with four columns: Flat_Price, BHK, Location, and Total_Sq.ft. The raw file contains 3,968 rows. Two rows missing a Location value are dropped, leaving 3,966 rows entering the cleaning stage.

2. Feature Parsing

- **BHK** – strings such as "6 BHK" or "1 RK" are split on whitespace and the numeric portion is cast to float (the unit text, BHK vs. RK, is discarded).
- **Total_Sq.ft** – strings such as "4200 sq.ft" are parsed the same way into a float.
- **Location** – the ", Kolkata" suffix is stripped from every value, leaving 201 distinct neighborhood names.
- **Flat_Price** – strings such as "Rs 8.5 Cr" or "Rs 45.0 L" are parsed by a custom function into a single unit, Crores (Lakh values are divided by 100).

3. Outlier Removal

Three filtering passes are applied in sequence:

1. Drop rows where Total_Sq.ft ÷ BHK < 250 (physically implausible room sizes) – removes 298 rows (3,966 → 3,668).
2. Per-location price-per-sqft filter: for each location with more than 5 listings, compute the local mean and standard deviation of price-per-sqft, and drop rows falling outside ±2 standard deviations. Locations with 5 or fewer listings are left untouched, since a standard deviation isn't meaningful at that sample size – removes a further 167 rows (3,668 → 3,501).
3. Cap Total_Sq.ft at 17,000 to remove a small number of extreme outliers – removes 6 rows (3,501 → 3,495 final rows).

4. Train / Test Split

The cleaned dataset (3,495 rows) is split 80/20 with random_state=42, producing 2,796 training rows and 699 test rows. Feature columns are Total_Sq.ft, BHK, and Location; the target is Flat_Price.

5. Preprocessing Pipeline

A single ColumnTransformer applies a OneHotEncoder (handle_unknown='ignore') to the Location column, while BHK and Total_Sq.ft pass through unscaled. This transformer is wrapped into one sklearn Pipeline together with the regressor, so encoding and prediction are saved and loaded as a single object – no

separate encoder file is needed at inference time.

6. Model Evaluation

Three regressors were trained on the same pipeline and compared on the held-out test set:

Model	Hyperparameters	Reported "MAE"*	R^2
Linear Regression	defaults	3.638	0.715
Random Forest	n_estimators = 125	1.695	0.867
Extra Trees	n_estimators = 180	0.638	0.950

** The notebook computes mean_squared_error but labels and prints it as "Mean Absolute Error." These values are therefore MSE (in Cr²), not true MAE – the Extra Trees RMSE works out to roughly 0.80 Cr, not 0.64.*

7. Production Model

ExtraTreesRegressor (n_estimators=180, random_state=42, n_jobs=-1), paired with the same ColumnTransformer preprocessing, is refit on the entire cleaned dataset (3,495 rows) rather than only the training split – meaning no data is held out from the version that is actually shipped, and the evaluation metrics above describe a slightly different (smaller) fit of the model.

8. Serving & Deployment

The fitted Pipeline object (preprocessing step + regressor, bundled together) is pickled to final_production_model.pkl. The Flask backend loads this file once at startup. The /predict route builds a single-row DataFrame from the incoming JSON payload (Total_Sq.ft, BHK, Location) and calls .predict() on it directly – the pipeline handles one-hot encoding internally, so the Flask layer never needs its own encoding logic.

Pipeline flow: Raw JSON input (Total_Sq.ft, BHK, Location) -> ColumnTransformer (OneHotEncoder on Location, passthrough on BHK/Total_Sq.ft) -> ExtraTreesRegressor -> Predicted Flat_Price (Cr)